17.5 Context Engine 插件系统

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200k tokens，输出 ~7k tokens（本节）

17.5.1 为什么需要 Context Engine

在 9.4 节里，我们已经看到**上下文压缩**（Compaction）怎样解决长对话的上下文溢出问题。不过那套机制本质上仍然是"被动裁剪"：只有当上下文真的超限，或者已经逼近模型承受范围时，系统才开始压缩。它很实用，但它假设消息历史主要还是保存在原来的会话结构里，压缩只是一个补救动作。

OpenClaw 在 `v2026.3.9` 引入了**上下文引擎**（Context Engine），源码位于 `src/context-engine/` 。这个模块把"消息怎么存、怎么取、怎么组装、什么时候压缩"从原来的运行时流程里剥离出来，变成一个可插拔框架。也就是说，开发者不只是能调压缩参数，而是可以替换整套上下文管理策略。

这样做的价值很直接。你可以把消息同步到数据库里做持久化，可以在模型调用前接入**向量检索**（Vector Retrieval）只挑最相关的历史片段，也可以设计自己的摘要策略，让不同类型的消息采用不同的压缩方式。对于想把 OpenClaw 改造成研究平台或产品底座的人来说，这个抽象层非常关键。

> **衍生解释——可插拔框架（Pluggable Framework）**
>
> 可插拔框架的意思是：主程序先定义一组稳定接口，真正的实现可以在运行时替换。这样做的好处不是"代码更优雅"这么简单，而是把变化隔离到边界上。上下文管理策略经常和模型、存储、检索方案一起变化，把它做成插件，比把逻辑写死在主循环里更适合长期演化。

17.5.2 插件接口（ContextEngine Interface）

Context Engine 的核心是一组接口方法，定义在 `src/context-engine/types.ts` 。它不是只暴露一个 `compact()` ，而是覆盖了会话初始化、消息摄入、上下文组装、压缩，以及子代理生命周期等多个阶段。接口如下：

```
// src/context-engine/types.ts
export interface ContextEngine {
  readonly info: ContextEngineInfo;
  bootstrap?(params: { sessionId: string; sessionFile: string }): Promise<BootstrapResu
  ingest(params: { sessionId: string; message: AgentMessage; isHeartbeat?: boolean }):
  ingestBatch?(params: { sessionId: string; messages: AgentMessage[]; isHeartbeat?: boo
  afterTurn?(params: { sessionId: string; sessionFile: string; messages: AgentMessage[]
  assemble(params: { sessionId: string; messages: AgentMessage[]; tokenBudget?: number
  compact(params: { sessionId: string; sessionFile: string; tokenBudget?: number; force
  prepareSubagentSpawn?(params: { parentSessionKey: string; childSessionKey: string; tt
  onSubagentEnded?(params: { childSessionKey: string; reason: SubagentEndReason }): Pro
  dispose?(): Promise<void>;
}
```

把它放进运行时视角看，这组方法其实就是一个上下文生命周期表。OpenClaw 把不同阶段的责任拆开，让插件作者按需接管，而不是一次性重写整个 Agent Loop。

阶段	方法	触发时机	用途
初始化	bootstrap()	会话首次加载	导入历史消息、初始化存储
摄入	ingest()	每条消息后	持久化消息到自定义存储
组装	assemble()	模型调用前	在 token 预算下选择消息
压缩	compact()	上下文溢出时	减少 token 使用
转向后	afterTurn()	运行尝试完成后	触发后台压缩、持久化
子代理	prepareSubagentSpawn()	子代理启动前	准备隔离的子会话状态
子代理结束	onSubagentEnded()	子代理完成后	清理或合并子会话

这里有两个点特别重要。第一， `assemble()` 和 `compact()` 是分开的：前者决定"本次送给模型什么"，后者决定"长期历史怎么缩减"。第二， `afterTurn()` 让引擎可以在一轮结束后做异步维护工作，这意味着上下文管理不再只是请求前的一次性拼装，而是变成持续演化的状态机。

`assemble()` 的返回值里还有一个很有意思的字段： `systemPromptAddition` 。这表示上下文引擎不仅能筛选消息，还能动态追加一段系统提示词，把检索结果、摘要结论或运行时约束直接注入模型调用。换句话说，Context Engine 管的不只是消息列表，也在影响系统提示词的最终形态。

17.5.3 注册与解析机制

OpenClaw 没有把 Context Engine 写成硬编码分支，而是用了很典型的**工厂模式**（Factory Pattern）配合全局注册表。注册表只保存 `id -> factory` 的映射，真正需要时才实例化引擎，这样异步初始化数据库连接、向量索引或远程存储就都变得可行。

```
// src/context-engine/registry.ts
export type ContextEngineFactory = () => ContextEngine | Promise<ContextEngine>;

export function registerContextEngine(id: string, factory: ContextEngineFactory): void
  getContextEngineRegistryState().engines.set(id, factory);
}

export async function resolveContextEngine(config?: OpenClawConfig): Promise<ContextEng
  const slotValue = config?.plugins?.slots?.contextEngine;
  const engineId = typeof slotValue === "string" && slotValue.trim()
    ? slotValue.trim()
    : defaultSlotIdForKey("contextEngine");  // 默认 "legacy"
  const factory = getContextEngineRegistryState().engines.get(engineId);
  if (!factory) throw new Error(`Context engine "${engineId}" is not registered`);
  return factory();
}
```

配置层面，它通过插件插槽来决定当前启用哪个引擎。用户只要在 `openclaw.json` 里改一个名字，运行时就会解析对应实现：

```
// openclaw.json
{
  "plugins": {
    "slots": {
      "contextEngine": "my-custom-engine"
    }
  }
}
```

这里采用的是**独占插槽**（Exclusive Slot）设计。同一时刻只能有一个 Context Engine 处于活跃状态；当你选择新引擎时，旧引擎会被自动挤出这个槽位。对上下文管理来说，这样的约束很合理，因为如果两个引擎同时声称自己负责组装和压缩，会立刻出现状态冲突。

> **衍生解释——工厂模式（Factory Pattern）**

> 工厂模式不是单纯把 `new` 包起来，而是把"如何创建对象"从"如何使用对象"里分离出去。对插件系统来说，这很重要，因为有些插件创建成本高，有些需要异步准备资源。注册工厂后，核心运行时无需知道实现细节，只关心最后拿到的接口对象是否符合约定。

17.5.4 LegacyContextEngine 向后兼容

新框架上线时，最怕的是把旧逻辑全打碎。OpenClaw 的做法很稳：它先提供一个 `LegacyContextEngine` ，把原有行为包进新接口里，当作默认引擎注册。这让旧版本会话管理和 9.4 节讲过的压缩流程几乎不用改，就能接入 Context Engine 框架。

```
// src/context-engine/legacy.ts
export class LegacyContextEngine implements ContextEngine {
  readonly info = { id: "legacy", name: "Legacy Context Engine", version: "1.0.0" };
  async ingest() { return { ingested: false }; }  // 无操作 — SessionManager 处理持久化
  async assemble(params) { return { messages: params.messages, estimatedTokens: 0 }; }
  async compact(params) {
    const { compactEmbeddedPiSessionDirect } = await import("../agents/pi-embedded-runn
    return compactEmbeddedPiSessionDirect({ ...params.runtimeContext, ...params });
  }
}
```

这段代码非常能说明设计意图。 `ingest()` 什么都不做，因为旧流程里消息持久化还是由 `SessionManager` 负责； `assemble()` 直接把消息原样返回，表示仍然沿用既有的消息整理管线； `compact()` 则委托给原来的压缩实现，也就是我们在 9.4 节已经分析过的那套逻辑。

因此， `legacy` 既是默认引擎，也是兼容层。只要没有在配置中显式切换，OpenClaw 的行为就和引入 Context Engine 之前保持一致。这种"先包一层适配器，再开放替换点"的做法，正是大型系统重构时常见的安全路径。

17.5.5 运行时集成

理解接口之后，还要看它怎样嵌进 Agent 的主循环。OpenClaw 在运行路径中按顺序接入了 Context Engine：先注册内置引擎，再解析活跃实现，然后在会话加载、模型调用和溢出恢复等关键点调用对应 hook。

具体来说，集成顺序是这样的。第一步， `run.ts` 调用 `ensureContextEnginesInitialized()` 注册内置引擎；第二步，紧接着调用 `resolveContextEngine(config)` 得到当前活跃实例；第三步，会话文件首次加载时在 `attempt.ts` 中执行 `bootstrap()` ；第四步，模型真正调用前执行 `assemble()` ；第五步，一旦检测到上下文溢出，就回到 `run.ts` 里调用 `compact()` 。

这个顺序很有讲究。 `bootstrap()` 负责把历史状态接进来， `assemble()` 负责把本轮需要的上下文挑出来， `compact()` 则只在预算压力出现时介入。这样一来，开发者可以单独替换其中某一层策略，而不必接管整条链路。

`attempt.ts` 里还有一个细节值得你记住： `assemble()` 的结果如果带有 `systemPromptAddition` ，运行时会把它前置到系统提示词之前。也就是说，Context Engine 可以把"消息选择"和"系统规则补充"合并成一次上下文准备动作，这给检索增强、动态约束注入、会话摘要提示都留出了空间。

`afterTurn()` 也在一轮运行结束后被调用。如果插件实现了它，运行时就把消息快照、已有消息数、token 预算和运行时上下文一起传进去；如果没有实现，系统才退回到逐条 `ingest()` 或批量 `ingestBatch()` 的后备逻辑。这里能看出 OpenClaw 的取向：优先给高级引擎完整控制权，但同时保留渐进接入路径。

17.5.6 开发自定义 Context Engine

从插件作者视角看，接入门槛并不高。你需要声明一个 `kind: "context-engine"` 的插件，并在 `register(api)` 里调用注册接口，把自己的实现挂到某个 id 上：

```
export default {
  id: "my-context-engine",
  name: "My Context Engine",
  kind: "context-engine",
  register(api) {
    api.registerContextEngine("my-engine", () => new MyContextEngine());
  }
};
```

最简单的自定义版本，可以先只实现 `assemble()` 和 `compact()` 。前者决定送给模型哪些消息，后者负责在预算紧张时怎样瘦身；等你需要把历史迁移到 SQLite、PostgreSQL 或向量数据库时，再逐步补上 `bootstrap()` 、 `ingest()` 和 `afterTurn()` 。

如果你的系统会频繁启用子代理，那么 `prepareSubagentSpawn()` 和 `onSubagentEnded()` 也值得实现。它们提供了主会话与子会话之间的上下文隔离点，让你能提前准备子会话状态，或在子代理结束后做清理、归档、摘要回写。这对多代理协作尤其重要，因为子代理往往会生成大量中间过程，未必应该原样回灌主上下文。

你可以把 Context Engine 看成 OpenClaw 上下文层的"总调度器"。9.1 到 9.4 节分别讲了系统提示词、工作区注入、身份系统和压缩机制，而到了这一节，这些能力第一次拥有了统一的可替换边界。它不是替代前几节的机制，而是把它们重新组织进一个更开放的运行时接口里。

本节小结

1. Context Engine 是 OpenClaw 在 `v2026.3.9` 引入的可插拔上下文管理框架，它把消息摄入、组装、压缩和子代理上下文管理统一抽象为接口。
2. 与 9.4 的被动压缩不同，Context Engine 提供了更高层的控制面，允许开发者接入检索增强、数据库持久化和自定义摘要策略。
3. `registerContextEngine()` 与 `resolveContextEngine()` 构成了基于工厂和注册表的解析机制，配置通过 `plugins.slots.contextEngine` 选择活跃实现。
4. `LegacyContextEngine` 把旧的 `SessionManager + compaction` 流程封装进新接口，保证默认行为与旧版本保持兼容。
5. 在运行时中， `bootstrap()` 、 `assemble()` 、 `afterTurn()` 与 `compact()` 分别嵌入会话加载、模型调用前后和溢出恢复阶段，其中 `systemPromptAddition` 还能动态补充系统提示词。